

GROCERY FINANCE SYSTEM

Intelligent Retail & Wholesale Order Management + AI Analytics Platform

College Project | Data Science + Neural Network + Deep Learning Labs

1. README

Project Name

Grocery Finance System

Description

The Grocery Finance System is a full-stack Retail & Wholesale Order Management web application with integrated AI/ML analytics. It supports two customer types (retail and wholesale/bulk buyers) alongside an owner admin dashboard with real-time sales reports, payment tracking, and AI-powered demand forecasting, anomaly detection, and customer segmentation.

The platform is structured across three academic AI lab modules — Data Science, Neural Network, and Deep Learning — making it a complete semester project for B.Tech / BCA / MCA / BSc Computer Science students.

Key Features

- Retail + Wholesale dual-pricing order management
- Owner admin dashboard: orders, products, customers, payments, reports
- JWT-based secure authentication for Owner and Customers
- Sales, profit, and order analytics powered by Chart.js
- Lab 1 (Data Science): EDA, K-Means customer segmentation, correlation heatmaps
- Lab 2 (Neural Network): MLP demand prediction, stock alert system, category classifier
- Lab 3 (Deep Learning): LSTM 7/30-day forecast, Autoencoder fraud & anomaly detection
- Streamlit + Gradio AI dashboards for interactive model exploration
- REST API built with FastAPI + Uvicorn (Python 3.10+)
- SQLite for development / MySQL for production deployment

Tech Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Bootstrap 5, Chart.js, JS	Responsive UI & animated charts
Backend	Python 3.10+, FastAPI, Uvicorn, Pydantic	REST API server
Authentication	JWT (jose), Bcrypt	Secure login tokens & password hashing
Database	MySQL (Production), SQLite (Dev/Demo)	All data storage
AI/ML	TensorFlow/Keras, Scikit-learn, Pandas, NumPy	Lab AI modules
Visualization	Matplotlib, Seaborn, Plotly, Streamlit, Gradio	Charts & AI dashboards
Icons	Font Awesome	UI icon library

User Roles

Role	Capabilities
Owner (Admin)	View all orders, customers, payments, reports. Add/edit products & stock. Confirm/reject orders. View retail vs. wholesale profit comparison. Access AI demand predictions and anomaly alerts.
Retail Customer	Register and login. Browse products at retail prices. Place small-quantity orders. Track own order status. View payment history.
Wholesale / Bulk Buyer	Register and login (marked as wholesale). Browse at wholesale prices. Place large bulk orders. Track order status. View credits and outstanding dues.

Installation & Quick Start

Step 1 — Install Python 3.10+

```
python --version # must be 3.10 or higher
```

Step 2 — Clone the repository

```
git clone https://github.com/yourusername/GroceryFinanceSystem.git
cd GroceryFinanceSystem
```

Step 3 — Create virtual environment & install packages

```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 4 — Configure MySQL database in backend/config.py

```
DATABASE_URL = "mysql+pymysql://root:yourpassword@localhost/grocery_db"  
SECRET_KEY   = "your-secret-key-here"
```

Step 5 — Create MySQL database and run schema

```
mysql -u root -p < backend/database/schema.sql
```

Step 6 — Start the FastAPI backend

```
uvicorn backend.main:app --reload --port 8000  
# API docs at: http://localhost:8000/docs
```

Step 7 — Open the website

```
# Open in browser: http://localhost:8000/pages/index.html
```

Step 8 — (Optional) Launch AI Streamlit Dashboard

```
streamlit run ai_features/streamlit_dashboard.py  
# Opens at: http://localhost:8501
```

2. File Structure

```
GroceryFinanceSystem/
├── README.md
├── requirements.txt
├── run.py                                ← Start full project from one command
├── website/                             ── MAIN WEBSITE ──
│   ├── pages/
│   │   ├── index.html                  ← Home / Landing page
│   │   ├── login.html                 ← Login page (Owner + Customer)
│   │   ├── register.html              ← New customer registration
│   │   ├── dashboard.html             ← Owner admin control panel
│   │   ├── place_order.html           ← Place new order (retail/wholesale)
│   │   ├── orders.html                ← View and manage all orders
│   │   ├── products.html              ← Product catalog + stock levels
│   │   ├── customers.html             ← All customers (retail + wholesale)
│   │   ├── payments.html              ← Payments, credit, dues
│   │   └── reports.html               ← Profit and sales reports with charts
│   ├── css/
│   │   └── style.css                  ← All styling for every page
│   └── js/
│       ├── api.js                    ← All fetch() calls to backend API
│       ├── auth.js                   ← Login, logout, token handling
│       ├── orders.js                 ← Order placing and tracking logic
│       ├── products.js               ← Product listing and filtering
│       └── charts.js                 ← All Chart.js graphs
├── backend/                             ── BACKEND API ──
│   ├── main.py                       ← FastAPI app entry point
│   ├── config.py                     ← Settings (DB path, secret key)
│   ├── database/
│   │   ├── db.py                     ← MySQL / SQLite connection
│   │   ├── schema.sql                ← All CREATE TABLE scripts
│   │   └── grocery.db                ← SQLite file (auto-created for dev)
│   ├── models/
│   │   ├── user.py                   ← User table model
│   │   ├── product.py                ← Product table model
│   │   ├── order.py                  ← Order table model
│   │   └── payment.py                 ← Payment table model
│   └── routes/
│       ├── auth.py                   ← /register /login /logout
│       ├── orders.py                 ← /orders /place-order /update-order
│       ├── products.py               ← /products /add-product /update-stock
│       ├── customers.py              ← /customers /customer/{id}
│       └── payments.py               ← /payments /credit /dues
```

└─ reports.py	← /reports /profit /market-compare
└─ ai_features/	— LAB MODULES (AI Bonus) —
└─ data_science/	← Lab 1
└─ eda.py	← Sales charts, top products
└─ statistics.py	← Mean, variance, correlation
└─ segmentation.py	← K-Means customer grouping
└─ market_compare.py	← Retail vs Wholesale profit analysis
└─ neural_network/	← Lab 2
└─ demand_model.py	← MLP model architecture
└─ train.py	← Train and save model
└─ predict.py	← Predict product demand
└─ deep_learning/	← Lab 3
└─ lstm_forecast.py	← 7-day / 30-day sales forecast
└─ autoencoder.py	← Anomaly detection model
└─ alerts.py	← Suspicious order alerts
└─ streamlit_dashboard.py	← AI results visual dashboard

3. Database SQL — MySQL

All queries below use MySQL. For development, SQLite can be used (change DATABASE_URL in config.py). For college demo and production deployment, use MySQL.

3.1 Create Database

```
CREATE DATABASE IF NOT EXISTS grocery_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE grocery_db;
```

3.2 Table: users

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100)          NOT NULL,
  email        VARCHAR(150)         NOT NULL UNIQUE,
  password     VARCHAR(255)         NOT NULL,
  role         ENUM('owner', 'retail', 'wholesale') NOT NULL DEFAULT 'retail',
  phone        VARCHAR(20),
  city         VARCHAR(100),
  created_at   DATETIME             NOT NULL DEFAULT
CURRENT_TIMESTAMP
);
```

3.3 Table: products

```
CREATE TABLE products (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(150)          NOT NULL,
  category     VARCHAR(100),
  retail_price DECIMAL(10, 2)       NOT NULL,
  wholesale_price DECIMAL(10, 2)   NOT NULL,
  stock_qty    INT                  NOT NULL DEFAULT 0,
  image        VARCHAR(255),
  created_at   DATETIME             NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

3.4 Table: orders

```
CREATE TABLE orders (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT                  NOT NULL,
  market_type ENUM('retail', 'wholesale') NOT NULL,
  total_amount DECIMAL(10, 2)       NOT NULL,
```

```

    status          ENUM('pending','confirmed','delivered','cancelled')
                                NOT NULL DEFAULT 'pending',
    created_at       DATETIME                                NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES users(id) ON DELETE CASCADE
);

```

3.5 Table: order_items

```

CREATE TABLE order_items (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    order_id        INT                NOT NULL,
    product_id      INT                NOT NULL,
    quantity        INT                NOT NULL,
    unit_price      DECIMAL(10,2) NOT NULL, -- price locked at time of order
    subtotal        DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE RESTRICT
);

```

3.6 Table: payments

```

CREATE TABLE payments (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    order_id        INT                NOT NULL,
    amount_paid     DECIMAL(10,2) NOT NULL,
    amount_due      DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    payment_type    ENUM('cash', 'credit', 'online') NOT NULL DEFAULT 'cash',
    payment_date    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE
);

```

3.7 Indexes for Performance

```

CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_status   ON orders(status);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_order_items_prod ON order_items(product_id);
CREATE INDEX idx_payments_order  ON payments(order_id);
CREATE INDEX idx_products_category ON products(category);

```

3.8 Useful Query Examples

Profit report — Retail vs Wholesale:

```

SELECT
    o.market_type,
    COUNT(o.id)           AS total_orders,
    SUM(o.total_amount)   AS total_revenue,
    SUM(p.amount_paid)    AS total_collected,

```

```
        SUM(p.amount_due)      AS total_pending_dues
FROM orders o
JOIN payments p ON p.order_id = o.id
GROUP BY o.market_type;
```

All pending orders with customer name:

```
SELECT o.id, u.name, u.role, o.market_type,
       o.total_amount, o.status, o.created_at
FROM orders o
JOIN users u ON u.id = o.customer_id
WHERE o.status = 'pending'
ORDER BY o.created_at DESC;
```

Top 5 best-selling products:

```
SELECT p.name, p.category, SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN products p ON p.id = oi.product_id
GROUP BY oi.product_id
ORDER BY units_sold DESC
LIMIT 5;
```

Monthly revenue summary:

```
SELECT DATE_FORMAT(created_at, '%Y-%m') AS month,
       COUNT(id) AS total_orders,
       SUM(total_amount) AS monthly_revenue
FROM orders
WHERE status != 'cancelled'
GROUP BY month
ORDER BY month DESC;
```

Customers with outstanding credit dues:

```
SELECT u.name, u.email, u.role,
       SUM(p.amount_due) AS total_dues
FROM payments p
JOIN orders o ON o.id = p.order_id
JOIN users u ON u.id = o.customer_id
WHERE p.amount_due > 0
GROUP BY u.id
ORDER BY total_dues DESC;
```


4. API Routes

Method	Route	Who Uses	Description
POST	/register	Customer	Create new customer account
POST	/login	Everyone	Login and receive JWT token
GET	/products	Everyone	View all products with prices
POST	/products/add	Owner	Add a new product
PUT	/products/{id}	Owner	Update product stock or price
POST	/orders/place	Customer	Place a new retail or wholesale order
GET	/orders	Owner	View all orders
GET	/orders/my	Customer	View own order history
PUT	/orders/{id}/status	Owner	Update order status
GET	/customers	Owner	View all registered customers
GET	/payments	Owner	View all payment records
POST	/payments/credit	Owner	Add a credit entry for a customer
GET	/reports/profit	Owner	View profit report by market type
GET	/reports/market	Owner	Retail vs Wholesale comparison
GET	/docs	Developer	Auto-generated FastAPI documentation

5. requirements.txt

```
# Backend
fastapi
uvicorn[standard]
sqlalchemy
pymysql
python-jose[cryptography]
bcrypt
python-multipart
pydantic
python-dotenv

# Data Science & AI
pandas
numpy
matplotlib
seaborn
plotly
```

```
scikit-learn  
tensorflow  
  
# Web AI Dashboards  
streamlit  
gradio
```